Atomics, CAS and Lock-Free Concurrency

9 min read Updated 28 Jul 2026 Topic 43 of 65

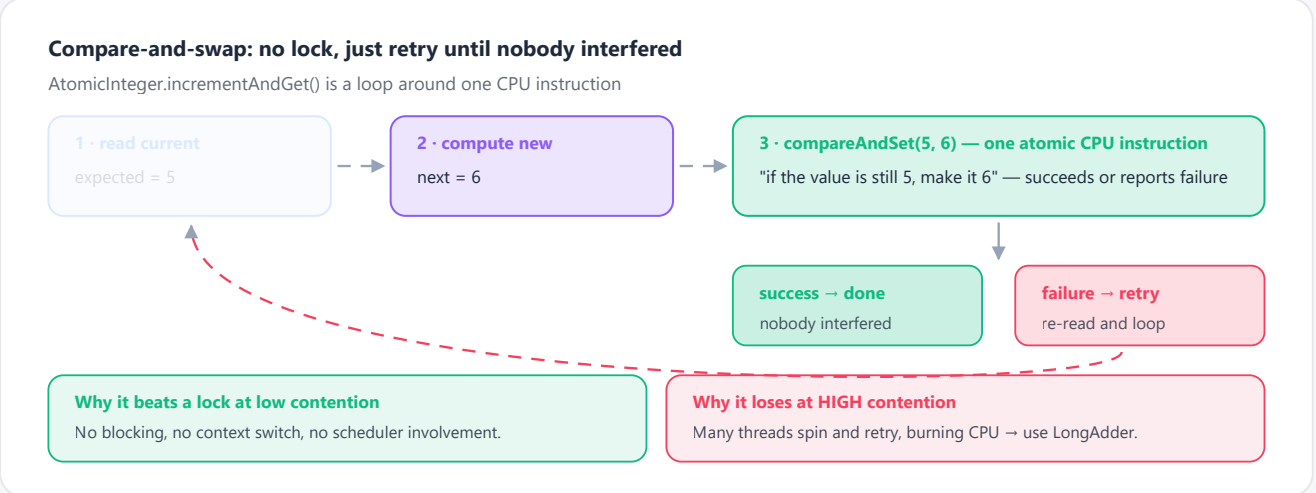
Why it matters

Atomics are the layer between "no synchronisation" and "a lock", and they are how the fastest concurrent code in Java is written — `ConcurrentHashMap`, the executors, `AQS` itself are all built on them. Interviewers ask about compare-and-swap because it explains how lock-free code is possible at all, and because knowing when CAS is *worse* than a lock shows judgement rather than enthusiasm.

The concept, simply

An atomic class wraps a value and offers operations that are indivisible without any lock. `AtomicInteger.incrementAndGet()` fixes the `count++` race from earlier in this chapter with no blocking at all.

It works with **compare-and-swap**, a single CPU instruction meaning "if this location still holds the value I expect, replace it; otherwise tell me you failed":



The retry loop is the whole idea. Nobody blocks — a thread that loses the race simply re-reads and tries again, which is fast when contention is low and wasteful when it is high.

The classes worth knowing:

Class	Use
`AtomicInteger` / `AtomicLong`	Counters, sequence numbers
`AtomicBoolean`	One-shot flags — "has this run?"
`AtomicReference<T>`	Lock-free swapping of an object
`LongAdder` / `DoubleAdder`	**High-contention counters** — much faster than `AtomicLong`
`AtomicStampedReference`	Adds a version stamp to defeat the ABA problem

CAS beats locking at low contention because nothing blocks: no context switch, no scheduler involvement, no parked threads. **It loses at high contention**, because many threads spin and retry, burning CPU on work that is thrown away. That is exactly the trade-off `LongAdder` addresses.

Code examples

JAVA

Copy

```java
// The race from earlier, fixed without a lock:
private final AtomicInteger count = new AtomicInteger();
count.incrementAndGet();          // atomic read-modify-write

// Atomic compute — the retry loop, written for you:
AtomicInteger max = new AtomicInteger();
max.updateAndGet(current -> Math.max(current, newValue));
max.accumulateAndGet(newValue, Math::max);      // equivalent

// ⚠ The function may run MORE THAN ONCE if the CAS fails and retries,
//   so it must be pure — no side effects, no logging, no I/O.
```

```java
// One-shot initialisation, without synchronized:
private final AtomicBoolean started = new AtomicBoolean();
public void start() {
    if (started.compareAndSet(false, true)) {   // ✓ exactly one winner
        doStart();
    }
}
```

`LongAdder`, the answer to "our counter is a bottleneck":

JAVA Copy

```java
// ✗ Under heavy contention every thread CASes the SAME memory location.
//   They also fight over the same cache line, so the cores spend their ti
//   invalidating each other — false sharing.
private final AtomicLong requests = new AtomicLong();
requests.incrementAndGet();

// ✓ LongAdder keeps several internal cells and lets threads hit different
//   ones, summing only when read. Dramatically faster under contention.
private final LongAdder requests = new LongAdder();
requests.increment();
long total = requests.sum();      // ⚠ approximate if writes are in flight

// Rule of thumb: metrics and counters you write constantly and read rarel
// → LongAdder. A value you need exactly and immediately → AtomicLong.
```

The ABA problem, which is the classic follow-up:

JAVA Copy

```java
// CAS only asks "is the value still what I expect?" — not "has it changed
// and changed back?". Thread 1 reads A; thread 2 changes A→B→A; thread 1'
// CAS succeeds, even though the world moved underneath it. For a counter
// that is harmless; for a stack popping nodes it can corrupt the structur

// ✓ AtomicStampedReference pairs the value with a version that always
```

```
int[] stamp = new int[1];
Node current = top.get(stamp);
top.compareAndSet(current, current.next, stamp[0], stamp[0] + 1);
```

And the limit of atomics — when you need a lock instead:

JAVA Copy

```
// ✗ Two atomics are individually atomic and NOT atomic together.
private final AtomicInteger low = new AtomicInteger();
private final AtomicInteger high = new AtomicInteger();
// Another thread can observe low > high between the two updates.

// ✓ One atomic reference to an immutable object holding BOTH values —
//   the whole state changes in a single CAS.
private record Range(int low, int high) { }
private final AtomicReference<Range> range = new AtomicReference<>(new Ran
range.updateAndGet(r -> new Range(r.low(), r.high() + 1));

// This is the general technique: make the state immutable, then swap the
// whole thing atomically. If it does not fit that shape, use a lock.
```

Under the hood

Atomics compile down to `VarHandle` operations (Java 9+, previously `sun.misc.Unsafe`) which map onto the CPU's native CAS instruction — `lock cmpxchg` on x86, load-linked/store-conditional on ARM. There is no lock and no kernel involvement, which is the source of the performance.

Atomics also give volatile semantics. Every atomic read and write carries the same visibility and ordering guarantees as a `volatile` field, so `AtomicInteger` is *both* atomic and visible — a strict upgrade over `volatile int` when the value is ever incremented.

same line cause the cores to invalidate each other's caches on every write. `LongAdder`'s cells are padded onto separate lines (the `@Contended` annotation), which is why it scales where a single `AtomicLong` does not.

Lock-free is not wait-free. "Lock-free" guarantees the *system* makes progress — some thread always succeeds. It does not guarantee any *individual* thread does: an unlucky thread can retry indefinitely while others keep winning. That is a real distinction, and a good precise answer to a question people usually answer loosely.

Common mistakes

- **Assuming several atomics are collectively atomic.** Each operation is; the sequence is not. Use one `AtomicReference` to an immutable object, or a lock.
- **Side effects inside `updateAndGet`.** The function can run several times on retry.
- `AtomicLong` **for a hot metric.** Use `LongAdder` under contention.
- **Ignoring ABA** in structures where identity matters. Use `AtomicStampedReference`.
- **Reading `get()` then writing `set()`.** That is a race again — use `compareAndSet` or `updateAndGet`.
- **Treating `LongAdder.sum()` as exact.** It is approximate while writes are in flight.
- **Reaching for atomics when a lock is clearer.** Complex invariants belong under a lock.

Interview lens

> **Q: "What is compare-and-swap?"**
> A: A single CPU instruction that atomically checks whether a memory location still holds an expected value and, if so, replaces it — reporting success or failure. Atomic classes wrap it in a retry loop: read the current value, compute the new one, attempt the swap, and start over if another thread got there first. No thread ever blocks.

A: Better at low to moderate contention, because nothing blocks — no context switch, no scheduler. Worse at high contention, because many threads spin and retry, burning CPU on work that gets discarded, whereas a lock would park them. It is also worse whenever the invariant spans several fields, since CAS is inherently single-location.

Q: "Why does `LongAdder` beat `AtomicLong`?"

A: `AtomicLong` has every thread CASing one memory location, so under contention they retry constantly and, because they share a cache line, invalidate each other's caches — false sharing. `LongAdder` keeps multiple padded cells and lets threads increment different ones, summing on read. So it is much faster to write and slightly more expensive to read, which suits metrics: written constantly, read occasionally.

Q: "What is the ABA problem?"

A: CAS only asks whether the value is still what you expected, not whether it changed and changed back. If a thread reads A, another changes it to B and back to A, the first thread's CAS succeeds even though the state moved underneath it. Harmless for a counter; genuinely dangerous for something like a lock-free stack, where a node may have been recycled. `AtomicStampedReference` solves it by pairing the value with a version that always increments.

Q: "Do atomics give you visibility as well?"

A: Yes — every atomic read and write has the same memory-model semantics as a `volatile` access. So `AtomicInteger` gives you visibility and atomicity, which makes it a strict improvement over `volatile int` whenever the value is updated based on its previous value.

A: Put them in one immutable object and hold a single `AtomicReference` to it, then swap the whole object with `updateAndGet`. Two separate atomics are individually atomic but not collectively, so another thread can observe an inconsistent pair between the updates. If the state does not fit that shape, that is the signal to use a lock instead.

Cheat-card

- **CAS** = "if still X, set to Y" in one CPU instruction, wrapped in a **retry loop**.
- Atomics give **atomicity AND volatile visibility**.
- Better than a lock at **low contention**; worse at high (spinning burns CPU).
- `LongAdder` **for hot counters** — padded cells beat one contended location; `sum()` is approximate.
- `updateAndGet` / `accumulateAndGet` functions must be **pure** — they can re-run.
- **ABA**: value returned to its old state undetected → `AtomicStampedReference`.
- Several atomics ≠ one atomic operation → **one** `AtomicReference` **to an immutable object**, or a lock.
- **Lock-free ≠ wait-free**: the system progresses; an individual thread may starve.
- False sharing: 64-byte cache lines, fixed with padding (`@Contended`).

QUICK CHECK — no marks, just for you.

1. How does AtomicInteger.incrementAndGet() work without a lock?

- ◯ It disables interrupts during the update
- ◯ It uses a ThreadLocal counter and sums on read
- ◯ It synchronises internally on the AtomicInteger object

A retry loop around compare-and-swap: read the current value, compute the next, attempt an atomic swap, and start over if another thread got there first

Check answer

2. Why does LongAdder outperform AtomicLong for a heavily-updated counter?

- ◯ It keeps multiple padded cells so threads increment different memory locations, summing only on read — avoiding both CAS retry storms and false sharing on one cache line
- ◯ It updates asynchronously on a background thread
- ◯ It uses a lock instead of CAS
- ◯ It stores the value as a double

Check answer

3. What is the ABA problem?

- ◯ Two threads updating the same value simultaneously
- ◯ A CAS succeeding because the value returned to its expected state after changing in between — the location looks untouched but the state moved underneath you
- ◯ An atomic operation being interrupted mid-instruction
- ◯ A deadlock between two atomic variables

Check answer

Prev
CompletableFuture and Asynchronous Composition

Next
Class Loading and the Delegation Model

LEARN MORE

ARTICLES & DOCS

- java.util.concurrent.atomic — package overview
  docs.oracle.com
- LongAdder — API docs
  docs.oracle.com

VIDEOS



The Volatile and Synchronized Keywords in Java | Atomic Variables | Java Multithreading | Geekific
Geekific

Hand-picked references for this lesson.